Table of Contents

Introduction 2
Requirements, Objectives, and Constraints 3
Requirements 3
Objectives 5
Constraints 5
Detailed Design and Implementation 6
Conceptual Design 7
Subsystem Designs 7
Decision Making 8
Design Parameters 9
Verification and Testing 10
Conclusions and Future Work 12
References 13
Appendices 14
Appendix A 14
Appendix B 17
Appendix C 17

Introduction

Maintaining balance is a fundamental challenge in robotics, especially for mobile systems with limited contact points like two-wheeled robots. Unlike four-wheeled vehicles that are naturally stable, two-wheeled robots must continuously adjust their position to stay upright, mimicking the behavior of an inverted pendulum. This dynamic instability presents a rich opportunity to explore control systems, sensor fusion, and real-time feedback loops. The goal of this project is to address this challenge by designing a robot that can self-balance in place and while in motion, using real-time sensor data and precise motor control.

In addition to balancing autonomously, the robot must be remotely controllable via Bluetooth, enabling a user to drive it forward, backward, and execute turns while maintaining stability. This requires the system to respond rapidly and accurately to external commands, integrating them seamlessly into its balancing algorithm. The robot must also navigate dynamic environments, including inclines, and recover from small disturbances, simulating real-world operating conditions. These requirements not only test the physical build quality of the robot but also its software architecture and control logic.

Solving this problem involves combining hardware components like IMUs, magnetic encoders, and motor drivers with software running on an embedded system. This integration is constrained by limited power and processing resources, making efficiency and precision critical. Developing a successful solution will demonstrate key skills in embedded systems, control theory, sensor interfacing, and system integration — all of which are fundamental to modern robotics and mechatronics.

Requirements, Objectives, and Constraints

The design of a self-balancing, two-wheeled robot involves meeting a set of functional requirements, performance objectives, and strict design constraints. The robot must autonomously maintain balance both while stationary and in motion, respond accurately to wireless Bluetooth commands, and navigate basic terrain features such as ramps and turns without falling. Beyond basic functionality, the system should optimize its stability, responsiveness, and recovery from disturbances. All of this must be achieved using only the hardware components and power source provided in the course kit, with software developed through the Arduino IDE on the Arduino Nano 33 BLE Sense Rev2 platform. These parameters define the scope and challenges of the project, guiding both the hardware implementation and control algorithm development.

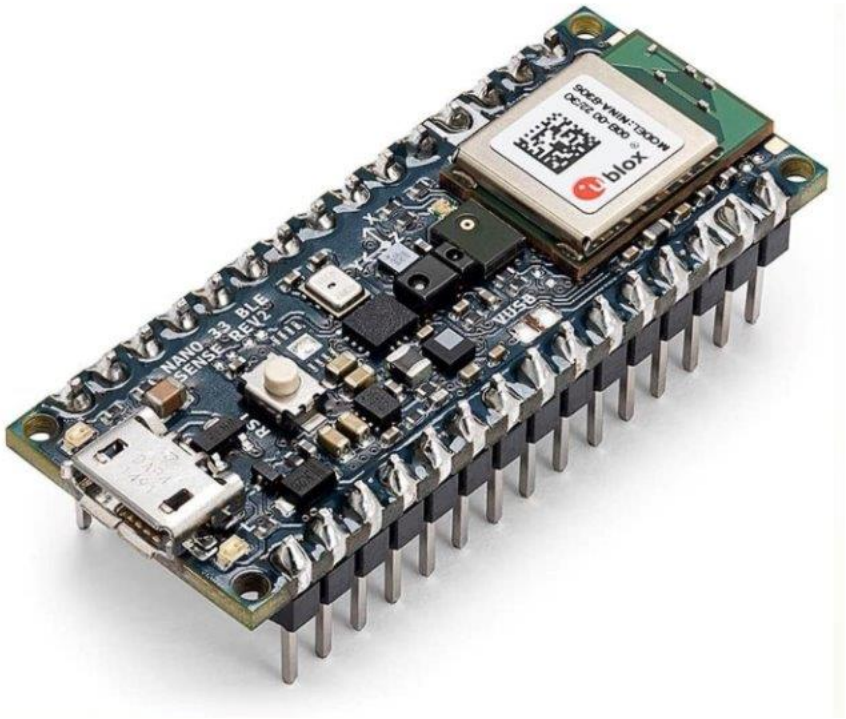


Figure 1: Arduino Nano 33 BLE Sense Rev2

Requirements

Category	**Requirement**	**Target/Condition**
Self-Balancing	Balance autonomously while stationary	Remain upright for ≥30 seconds
		Lateral oscillation within ±4 cm
		Average speed < 1 cm/s

Remote Control	Controlled via Bluetooth with four commands	Forward, Backward, Turn Left, Turn Right
	Forward-Backward movement	Move, then pause
	90° turn (Left and Right)	Two 45° turns with pauses
In-Motion Balance	Maintain balance while driving on flat surface	At least 50 cm forward/backward
		Speed ≥10 cm/s
Turning	Maintain balance while turning	Each 45° turn in ≤2 seconds
Ramp Navigation	Ascend, stay, and descend on ramp of 5, 10, and 15° planes	Angle ≥15°

Objectives

Optimization	Minimize stationary oscillations	Target: < ±4 cm
	Maximize ramp angle for autonomous movement and balance	Target: >15°
	Maximize tilt recovery angle on flat surface	Target: >15°

Constraints

Constraint Category	Description
Hardware Components	Only components from the course-provided kit may be used
Power Source	Limited to the battery pack included in the kit
Development Platform	Software must be developed using the Arduino IDE
Microcontroller	Must use the Arduino Nano 33 BLE Sense Rev2
Sensor	Must use the onboard BMI270 BMM150 IMU sensor
Motor Driver	Must use the DRV8833 dual H-bridge motor driver
Motors	Must use Pololu 4741 DC motors provided in the kit
Financial	Limited budget based on personal finances
Temporal	Short development timeline of a few weeks

Detailed Design and Implementation

The final design is a self-balancing, two-wheeled robot built around a 3D-printed chassis and powered by 8 rechargeable 1.5V batteries, with an Arduino Nano 33 BLE Sense Rev2 serving as the central controller. It maintains balance using a complementary filter that fuses gyroscope and accelerometer data to estimate tilt, feeding this into a PID controller that generates PWM signals to drive two DC motors through H-bridge drivers. Wheel velocity is measured using two AS5600 magnetic encoders, managed via a TCA9548A I2C multiplexer. The robot incorporates a cascaded control structure where velocity error is translated into a target tilt angle, allowing smooth forward and backward motion, while turning is achieved by asymmetrically adjusting motor speeds with empirically tuned correction factors. A Flutter Bluetooth app and serial keyboard interface support real-time testing and parameter tuning. The design was validated through Simulink modeling, sensor calibration, and extensive trial-and-error tuning of PID gains and motion logic, resulting in a stable and responsive system capable of dynamic balancing and directional control.

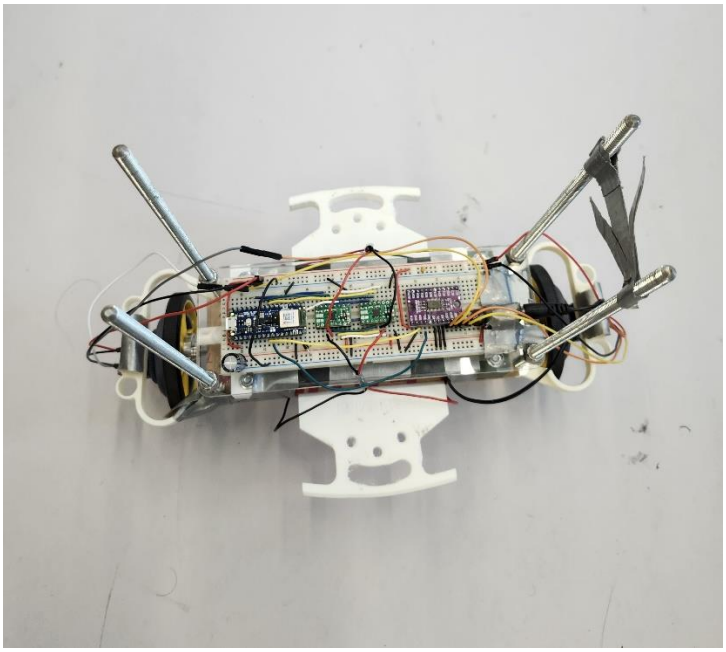


Figure 2: Completed Robot

Conceptual Design

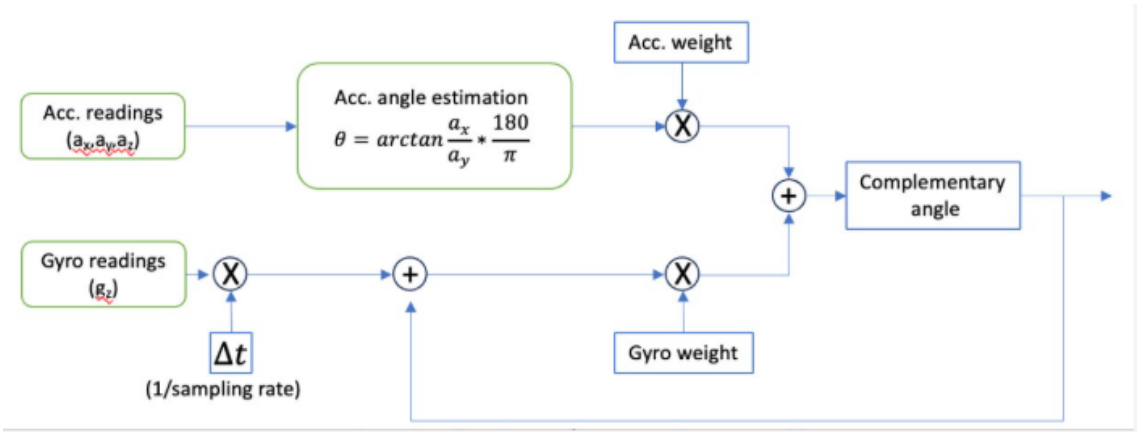


Figure 3: Feedback control diagram

The self-balancing robot was conceptually designed as a dynamic feedback control system that maintains an upright position using sensor data and real-time actuation. At a high level, the robot architecture consists of a central microcontroller interfacing with onboard inertial sensors to estimate the robot's tilt angle. Figure 3 is diagram that approximates the IMU's sensor fusion in order to calculate our angle. A primary control loop uses this angle to compute a corrective response via a PID controller. Some of our rough-work and conceptual understanding of balancing can be found in Appendix A.

The response is then translated into PWM signals, which drive the motors to apply force in a direction that restores balance. In motion, the robot combines this balancing logic with velocity control and turning logic. These are implemented as supplementary layers atop the core balancing loop: velocity control adjusts the target tilt angle to induce movement, while turning logic asymmetrically adjusts motor speeds to produce curved trajectories. The system's modular architecture includes sensing, actuation, communication, and control layers, allowing flexible development and testing. The overall design objective was to maintain stability while enabling forward, backward, and turning motion.

Subsystem Designs

The key subsystems include the mechanical body, sensing and control electronics, actuation, and interface tools. Mechanically, the robot used a custom 3D-printed body, powered by 8 × 1.5V rechargeable batteries, selected for their high energy density and compact form. The Arduino Nano 33 BLE Sense Rev2 served as the central controller, chosen for its integrated Bluetooth and 9-axis IMU, which eliminated the need for external orientation sensors. For wheel velocity measurement, two AS5600 magnetic encoders were mounted on the wheel shafts. However, since both encoders shared the same I2C address, a TCA9548A I2C multiplexer was incorporated to allow separate

access to each sensor. Actuation was provided by two DC motors driven through H-bridge motor drivers, wired to allow full bidirectional control as in figure 4. The electrical layout included parallel connections of the drivers for higher current capacity and flexible motor command routing via the PWM-capable Arduino pins.

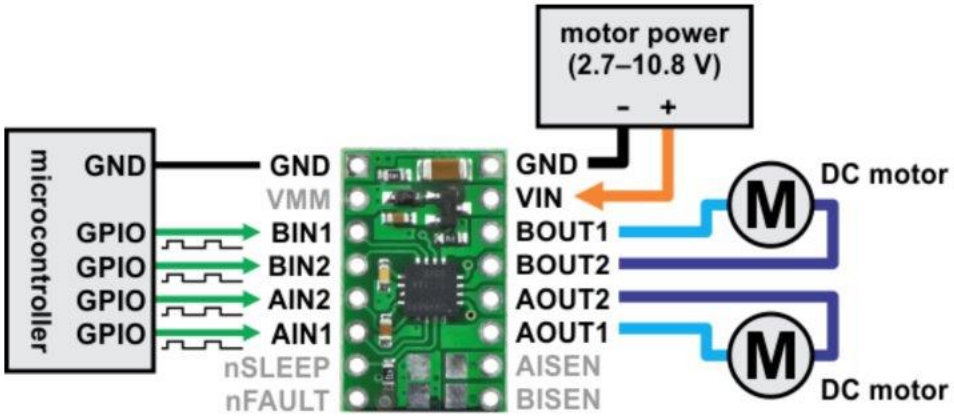


Figure 4: Diagram of H-bridge Setup

For system interaction and tuning, a Flutter-based mobile app was developed to send test commands over Bluetooth, allowing real-time control of movement and behavior. A separate keyboard interface was implemented to adjust PID gains through the serial monitor, avoiding the need to reflash firmware for each change. These tools formed the user interface subsystem and were critical for iterative tuning and testing. Additionally, the software stack included logic to fuse accelerometer and gyroscope readings via a complementary filter, providing a more stable and drift-free angle estimate. Code was written to manage forward/reverse motion and turning through dedicated PWM signal logic tailored to each motor channel.

Decision Making

Multiple design alternatives were explored during the development of the robot, with choices driven by empirical performance, simplicity, and integration ease. For velocity control, two major approaches were tested. The first directly blended the PID output for balance with a secondary PWM signal derived from velocity error. The second implemented a cascade structure where the velocity error informed a new desired angle, which was then fed into the primary angle PID controller. After comparative testing, the second approach was adopted due to its smoother motion and better integration with the balancing behavior, especially during acceleration and deceleration phases.

Hardware selection also involved trade-offs. The Arduino Nano 33 BLE Sense Rev2 was chosen over other microcontrollers for its compact form factor and built-in IMU and Bluetooth, reducing wiring complexity and increasing reliability. The decision to use AS5600 encoders was based on their precision and compact magnetic sensing,

though their shared I2C address required the added complexity of a TCA9548A multiplexer. Additional motor tuning decisions were made based on observed asymmetries between the left and right motors; specific scaling factors were introduced in software to correct these inconsistencies. Turning behavior introduced challenges due to the inherent imbalance it created in the system, prompting the team to manually adjust forward/reverse motion during turning to suppress swing-back effects. Each of these decisions reflects a careful balance of performance, feasibility, and complexity, validated through testing.

Design Parameters

A number of specific parameters and models were employed to validate and optimize the design. Motor control was achieved through PWM values ranging from 0 to 255, with direction determined by signal polarity and magnitude determined by the PID controller's output. The PID controller took the angular error (difference between actual tilt and desired zero-degree reference) and converted it into a control effort. Empirical PID tuning was performed using a methodical approach: increasing proportional gain (Kp) until oscillations occurred, then adding integral gain (Ki) to correct steady-state error, and finally introducing derivative gain (Kd) to dampen overshoot and stabilize response.

The sensor fusion system used a complementary filter to combine accelerometer and gyroscope data, taking advantage of the low-frequency stability of the accelerometer and the high-frequency smoothness of the gyroscope. This allowed for reliable angle estimation despite the noisy and drifting nature of each individual sensor. A Simulink model was also developed to simulate the robot's behavior and assist in initial tuning. Parameters like center of mass location, body length, and estimated inertia were input into the model to auto-tune rough PID values, which were then used as starting points for real-world testing.

To correct for physical asymmetry, calibration constants were introduced for each motor's forward and reverse behavior (e.g., turn_L = 1, turn_R = 1.1 or -1.15), determined through repeated test runs. These constants ensured that both wheels behaved consistently during straight motion and turns. Every time hardware changed—such as replacing a battery—retuning was necessary, emphasizing the sensitivity of the system to small parameter shifts. Overall, the design parameters were integral to achieving functional stability and performance and were refined through iterative simulation and real-world experimentation.

Verification and Testing

To ensure that the robot met the specified requirements for balancing, forward/backward movement, turning, and ramp climbing; we conducted a series of iterative parameter tuning experiments using our custom Bluetooth control app. This allowed us to dynamically adjust control parameters such as desired velocity, encoder PID gains, and offset angles during runtime.

1. Balancing

    After tuning the PID system of the robot, the robot consistently drifted to one side so we adjusted the offset angle slightly (typically by ±0.2°) rather than recalculating the entire PID loop to try and mitigate this effect.

2. Forward and Backward Motion

    To tune the robot's ability to move forward and backward smoothly while maintaining balance, we adjusted the desired velocity parameter which serves as the setpoint input for the PID controller. This value was incremented or decremented in steps of 0.1 cm/s to identify a range that allowed for controlled movement without overshoot or instability. The optimal velocity range provided steady motion without causing the robot to fall forward or backward.

3. Turning Behavior

    Turning was achieved by creating a differential velocity between the left and right wheels. We tuned the ratio between the wheel speeds to produce a smooth arc without abrupt swinging. In addition, we optimized two critical parameters:

        kp_en: This parameter adjusts the weight of encoder feedback in the control loop. A higher kp_en causes the robot to turn more aggressively but can lead to over-tilting or instability if set too high.

        desired_velocity: Higher turning speeds can lead to excessive drift and instability. We adjusted this in 0.1 increments to balance responsiveness with stability.

    We observed that when velocity was too high, the robot had difficulty stopping and would drift. Conversely, if kp_en was too high, the robot would over-rotate and destabilize. Fine-tuning both values allowed us to achieve smooth, controlled turns.

4. Ramp Climbing

    Ramp climbing required further adjustments to kp_en, as increased reliance on encoder feedback improved the robot's ability to climb but introduced oscillations on flat surfaces. This trade-off was acceptable for the climbing task. Additionally, we gradually increased the desired lean angle in steps of approximately 2° to

match the ramp inclinations of 5°, 10°, and 15°. We evaluated the robot's performance by observing its ability to climb without tipping or excessive oscillation.

5. Post-Collision and Power Cycle Testing

After any impact or battery recharge, the robot was re-evaluated to confirm that it maintained proper balance and did not exhibit constant drift or excessive oscillations. Inconsistent behavior across trials indicated the need for further parameter adjustments.

Offset tuning allowed us to compensate for mechanical or calibration imperfections without affecting core control stability.

6. Validation and Requirement Confirmation

For each motion behavior (balancing, movement, turning, ramp climbing), we conducted multiple trials to ensure repeatability and compliance with functional requirements. A set of tuned parameters was only accepted once consistent performance was demonstrated across several trials. If any test failed, parameters were re-adjusted, and testing was repeated until all conditions were reliably met.

Conclusions and Future Work

The project successfully met all major objectives, including achieving self-balancing within a ±2 cm range and maintaining stability while climbing a 10-degree ramp. Through the development process, the team gained valuable hands-on experience with sensor fusion, real-time control systems, and PID tuning. Key lessons included the importance of physical balance and symmetry in hardware, as even minor mechanical inconsistencies—such as uneven wheel behavior or shifting center of mass—required compensatory adjustments in software. A late-stage hardware failure due to a loose wheel underscored the need for thorough mechanical checks and robust debugging strategies, reinforcing the relationship between mechanical integrity and control performance.

To build on the current design, future improvements should focus on automating more aspects of the system. This includes cleaning and modularizing the codebase and integrating timers to enable predefined motion sequences, replacing manual Bluetooth toggling. Mechanically, improving the structural balance of the robot and securing all moving parts more effectively would enhance reliability and reduce the need for constant re-tuning. Additionally, implementing real-time feedback or alerts for hardware issues—such as wheel slippage—could help prevent late-stage failures. Further enhancements might also explore advanced control techniques or integrating obstacle avoidance for more autonomous and robust navigation.

References

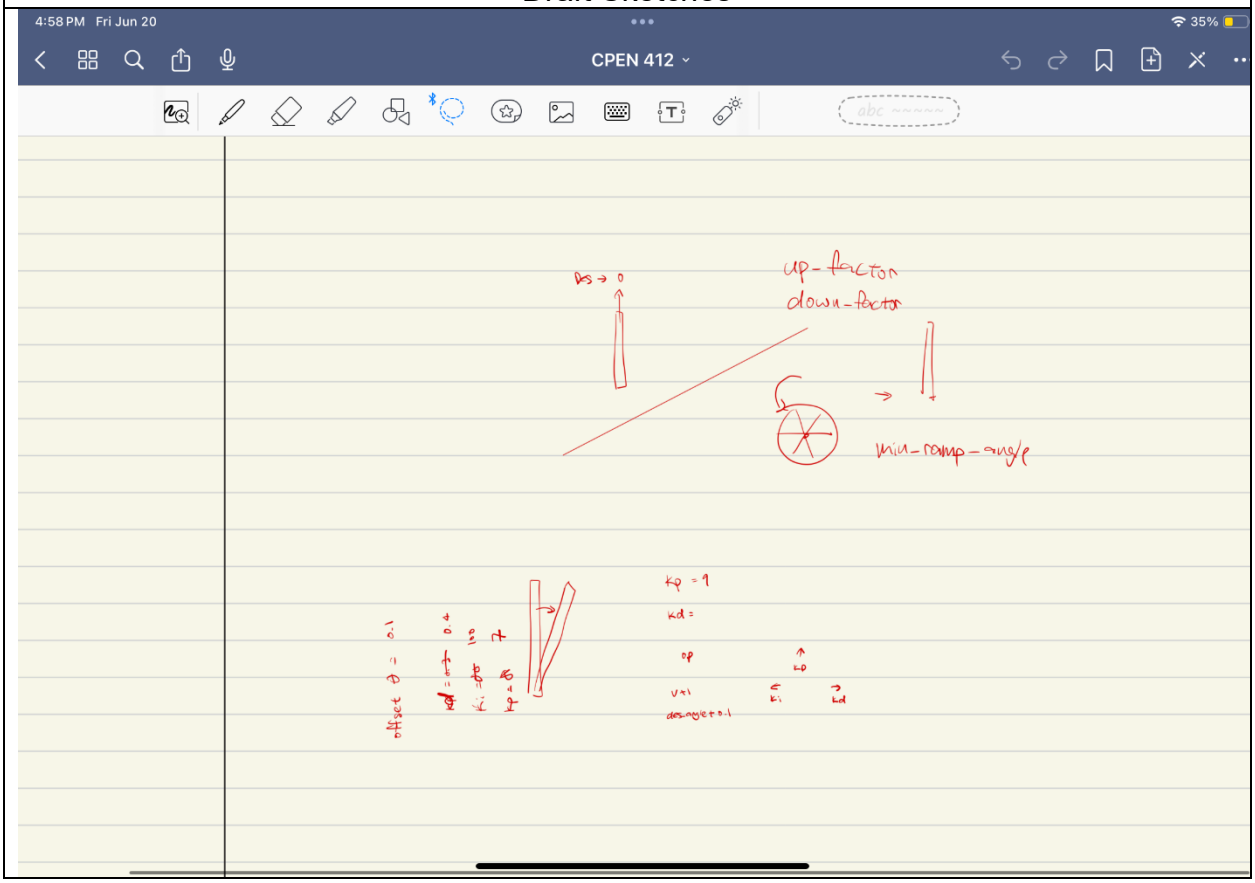
We used the given ELEC 391 resources from previous lessons and tutorials such as the Bluetooth flutter tutorial and inverted pendulum equations for balancing.

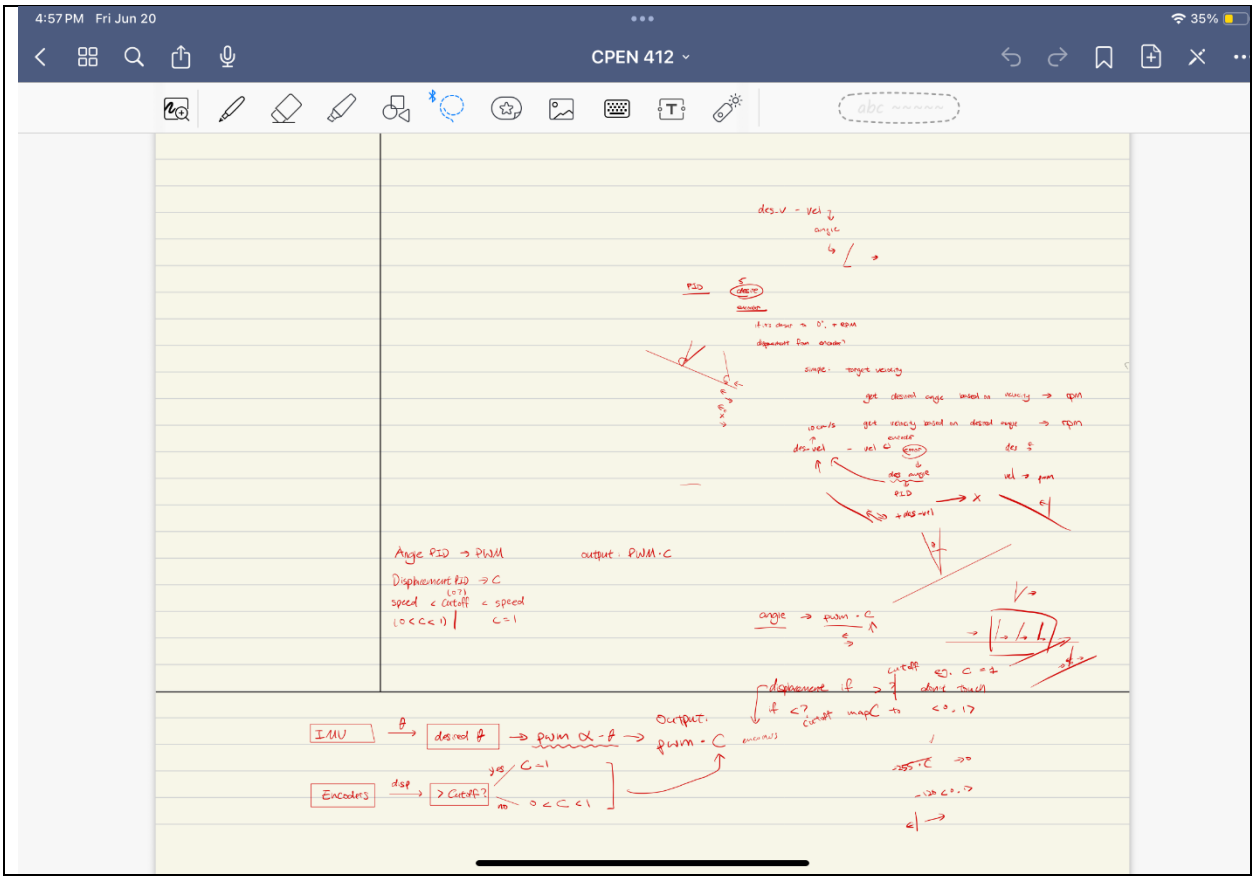
Appendices

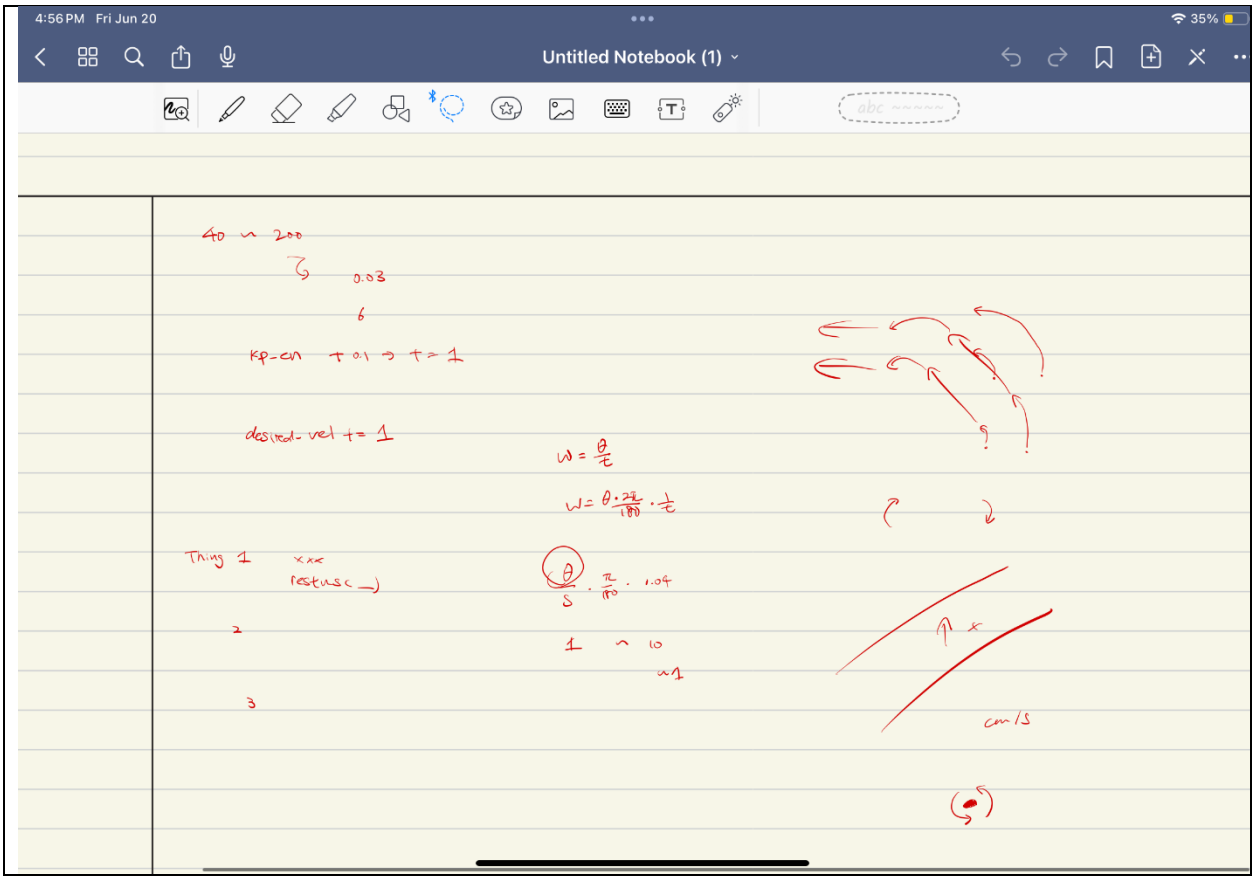
Include sections that do not fit well in the main body of the report but help to show the development progress and justify your decisions. These might include (but may not be restricted to) the following:

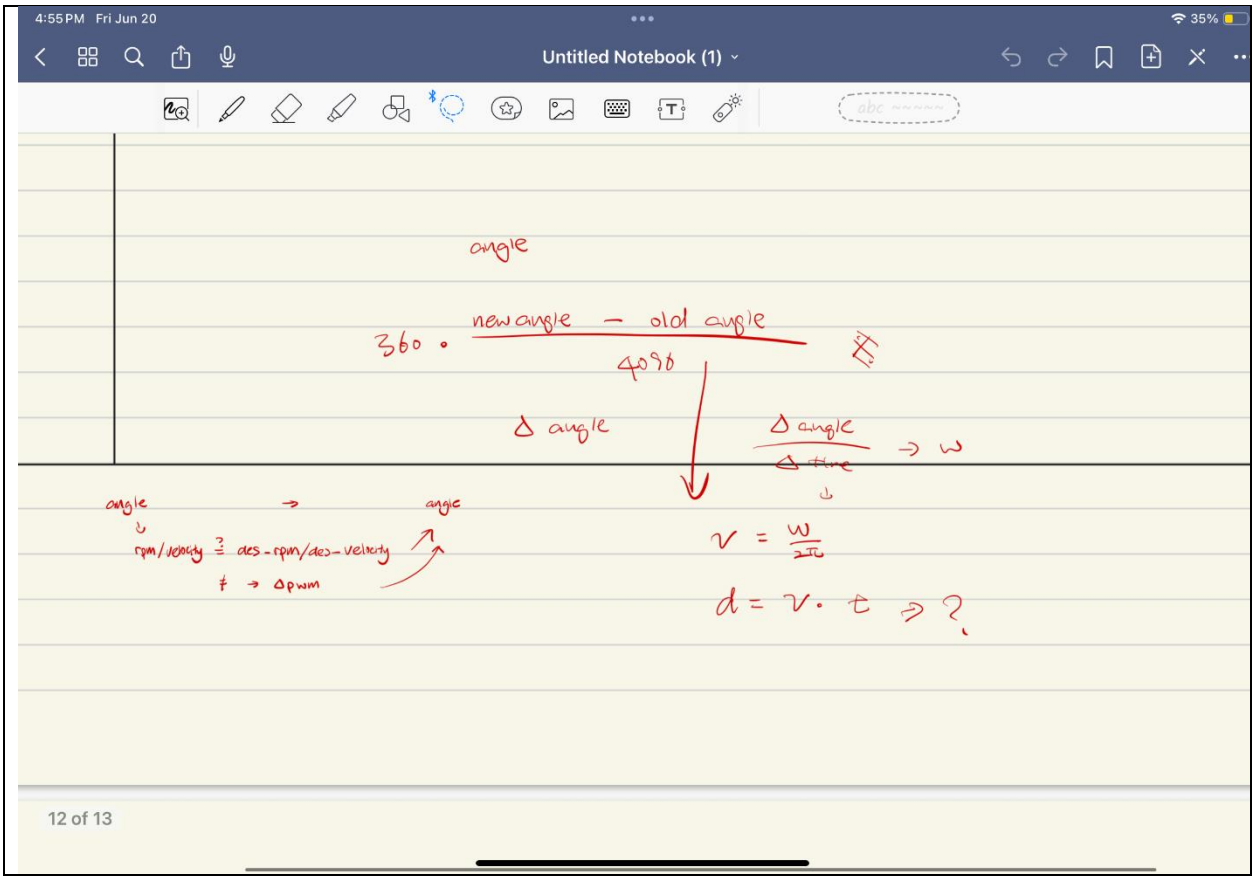
Appendix A

Draft Sketches




4:57 PM Fri Jun 20
35%
CPEN 412
Angle PID → PWM
output : PWM·C
Displacement PID → C
speed < Cutoff < speed
(0 < C < 1)
C = 1
IMU
desired θ
pwm α - β
Output:
pwm · C
Encoders
disp
> Cutoff?
yes
no
C = 1
0 < C < 1


4:56 PM Fri Jun 20
35%
Untitled Notebook (1)
40 ~ 200
0.03
6
kp-cm + 0.1 → t = 1
desired-vel t = 1
Thing 1
xxx
restwise
2
3
1 ~ 10
~1
cm/s



Appendix B

[Arduino® Nano 33 BLE Sense Rev2 Datasheet]

[DRV8833 Dual H-Bridge Motor Driver Datasheet]

[Pololu 4741 DC motors Datasheet]

[AS5600 Magnetic Encoders Datasheet]

[AS5600 Magnetic Encoders Library]

[TCA9548A I2C Multiplexer Datasheet]

[TCA9548A I2C Multiplexer Library]

[Arduino BMI270 BMM150 Library]

[Arduino BLE Library]

[Arduino Flutter Bluetooth Tutorial]

Appendix C

balance_working.ino
// A is left wheel and B is right wheel

```
// AIN1 and BIN1 used for reverse
// AIN2 and BIN2 used for forward

#include <AS5600.h>
#include <ArduinoBLE.h>
#include <Wire.h>
#include "TCA9548A.h"
#include <math.h>
#include <Arduino_BMI270_BMM150.h>
#include <string.h>

#define BUFFER_SIZE 20

TCA9548A I2CMux;
AS5600 encoderLeft;
AS5600 encoderRight;

// Define a custom BLE service and characteristic
BLEService customService("00000000-5EC4-4083-81CD-A10B8D5CF6ED");
BLECharacteristic customCharacteristic(
    "00000001-5EC4-4083-81CD-A10B8D5CF6ED", BLERead | BLEWrite | BLENotify,
BUFFER_SIZE, false);



// Define motor pins
#define BIN1 D3
#define BIN2 D5
#define AIN2 D6
#define AIN1 D9
#define A 1
#define B 2

float vel, angV0,angV1;
// Constants and state variables
float k = 0.9;
float kp = 7;
float ki = 100;
float kd = 0.4;
float kp_en = 0;
float ki_en = kp_en/200.0;

float offset_angle = 0.4; // balance setpoint
float desired_angle = 0; // balance setpoint
float desired_vel = 0;
float turn_L = 1;
```

```
float turn_R = 1;
float old_theta = 0;
float gyro_theta = 0;
float old_error = 0;
float old_error_en = 0;
float i_theta = 0;
float i_theta_en = 0;

int task = 0;
String input;
unsigned long start_time = 0;

// Motor control functions
void motorOff(int num){
  if(num == A){
    analogWrite(AIN1,255);
    analogWrite(AIN2,255);
  } else if(num == B){
    analogWrite(BIN1,255);
    analogWrite(BIN2,255);
  }
}

void forward(int num, int pwm){
  if(num == A){
    analogWrite(AIN1,255);
    analogWrite(AIN2,constrain(255 - pwm*turn_L, 0, 255));
  } else if(num == B){
    analogWrite(BIN1,255);
    analogWrite(BIN2,constrain(255-pwm*turn_R, 0, 255));
  }
}

void reverse(int num, int pwm){
  if(num == A){
    analogWrite(AIN1,constrain(255- pwm*turn_L, 0, 255));
    analogWrite(AIN2,255);
  } else if(num == B){
    analogWrite(BIN1,constrain(255- pwm*turn_R, 0, 255));
    analogWrite(BIN2,255);
  }
}

void both_motorOff(){
  motorOff(A);
  motorOff(B);
```

```
}

void both_forward(int pwm){
  forward(A, pwm);
  forward(B, int(pwm*0.99));
}

void both_reverse(int pwm){
  reverse(A, pwm);
  reverse(B, int(pwm*0.99));
}

void setup() {
  //Serial.begin(9600);
  //Serial.setTimeout(10);
  //while (!Serial);

  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU!");
    while (1);
  }
  Wire.begin();
  I2CMux.begin(Wire);
  I2CMux.closeAll();

  I2CMux.openChannel(0);
  delay(10);

  encoderLeft.begin(4);  //  set direction pin.
  encoderLeft.setDirection(AS5600_CLOCK_WISE);  //  default, just be explicit.
  encoderRight.begin(4);  //  set direction pin.
  encoderRight.setDirection(AS5600_CLOCK_WISE);  //  default, just be explicit.

  int b = encoderLeft.isConnected();
  Serial.print("Connect: ");
  Serial.println(b);

  int c = encoderRight.isConnected();
  Serial.print("Connect: ");
  Serial.println(c);

  pinMode(BIN1, OUTPUT);
  pinMode(BIN2, OUTPUT);
  pinMode(AIN1, OUTPUT);
  pinMode(AIN2, OUTPUT);
```

```
    // Initialize the built-in LED to indicate connection status
  pinMode(LED_BUILTIN, OUTPUT);

  if (!BLE.begin()) {
    Serial.println("Starting BLE failed!");
    while (1);
  }

  // Set the device name and local name
  BLE.setLocalName("Lance");
  BLE.setDeviceName("Lance");

  // Add the characteristic to the service
  customService.addCharacteristic(customCharacteristic);

  // Add the service
  BLE.addService(customService);

  // Set an initial value for the characteristic
  customCharacteristic.writeValue("Waiting for data");

  // Start advertising the service
  BLE.advertise();

  Serial.println("BluetoothÂ® device active, waiting for connections...");
  Serial.println("Setup Finished");
  start_time = micros();
}

void keyboard_test (void) {
    if (Serial.available() > 0) {
        input = Serial.readString();
        input.trim();
        input.toLowerCase();
    }

    if (input == "kp") {
        task = 1;
    }
    else if (input == "ki") {
        task = 2;
    }
    else if (input == "kd") {
        task = 3;
    }
    else if (input == "reset") {
```

```
        i_theta = 0;
    }
    else if (input == "kpen") {
      task = 4;
    }
     else if (input == "angle") {
      task = 5;
    }


    switch (task) {
        case 1:
            if(input == "0" || input.toFloat() > 0) kp = input.toFloat();
        break;
        case 2:
            if(input == "0" || input.toFloat() > 0) ki = input.toFloat();
        break;
        case 3:
            if(input == "0" || input.toFloat() > 0) kd = input.toFloat();
        break;
        case 4:
            if(input == "0" || input.toFloat() > 0) kp_en = input.toFloat();
        break;
        case 5:
            if(input == "0" || input.toFloat() > 0) offset_angle = input.toFloat();
        break;
    }
}

void loop() {
  float acc_x, acc_y, acc_z;
  float gyro_x, gyro_y, gyro_z;
  float acc_theta, theta, vel;
  float error, error_en, d_theta, pid_out, pid_out_imu, pid_out_en;
  int pwm;
  //Bluetooth setup
  // Wait for a BLE central to connect
  BLEDevice central = BLE.central();

  if (central) {
    //Serial.print("Connected to central: ");
    Serial.println(central.address());
    digitalWrite(LED_BUILTIN, HIGH); // Turn on LED to indicate connection

    // Keep running while connected
    while (central.connected()) {
```

```
keyboard_test();
//Serial.println("Connected from central.");
//Finish bluetooth setup
  if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
    // Read accelerometer
    IMU.readAcceleration(acc_x, acc_y, acc_z);
    acc_theta = atan2(acc_y, acc_z) * 180.0 / M_PI;

    // Read gyroscope
    IMU.readGyroscope(gyro_x, gyro_y, gyro_z);
    gyro_x = -gyro_x;

    // Calculate delta time
    unsigned long now = micros();
    float dt = (now - start_time) / 1000000.0;
    start_time = now;

    // Integrate gyro
    gyro_theta += gyro_x * dt;

    // Complementary filter
    theta = (1 - k) * acc_theta + k * (old_theta + gyro_x * dt);

    //Angular speed
    angV0 =encoderLeft.getAngularSpeed(AS5600_MODE_RPM);
    Serial.print("left encoder speed: ");
    Serial.println(angV0);
    I2CMux.closeChannel(0);
    I2CMux.openChannel(1);
    angV1 = encoderRight.getAngularSpeed(AS5600_MODE_RPM);
    Serial.print("right encoder speed: ");
    Serial.println(angV1);
    I2CMux.closeChannel(1);
    I2CMux.openChannel(0);
    vel =(-angV0 + angV1)/2;//in angle/sec
    Serial.print("angular speed: ");
    Serial.println(vel);
    vel = vel * PI / 180 * 4;// in cm/sec
    Serial.print("velocity: ");
    Serial.println(vel);

    //displacement into a capping(?) constant
    error_en = desired_vel - vel;
    Serial.print("velocity desired: ");
    Serial.println(desired_vel);
```

```
Serial.print("velocity error: ");
Serial.println(error_en);
i_theta_en += ki_en *(error_en + old_error_en) / 2.0 * dt;
Serial.print("kp_en: ");
Serial.println(kp_en);
Serial.print("i theta_en: ");
Serial.println(i_theta_en);
pid_out_en = (kp_en * error_en) + constrain(i_theta_en,-10,10);
Serial.print("output desired angle:  ");
Serial.println(pid_out_en);
desired_angle = constrain(pid_out_en,-15,15);

// PID calculations
error = (offset_angle + desired_angle) - theta;
d_theta = (old_theta - theta) / dt;
i_theta += ki *(error + old_error) / 2.0 * dt;
i_theta = constrain(i_theta,-50,50);
pid_out = (kp * error) + (i_theta) + (kd * d_theta);

if(pid_out < -255 or pid_out > 255) {
  pwm = 255;
}
else pwm = int(abs(pid_out));

//Serial.print("offset_angle: "); Serial.print(offset_angle);
//Serial.print("\t");
//Serial.print("Kp: "); Serial.print(kp);
//Serial.print("\t");
//Serial.print("Ki: "); Serial.print(ki);
//Serial.print("\t");
//Serial.print("Kd: "); Serial.print(kd);
//Serial.println("\t");
//Serial.print(" i_theta: "); Serial.print(i_theta);
//Serial.print("\t");
//Serial.print(" kd*d_theta: "); Serial.println(kd*d_theta);

// Apply motor control
if (pid_out < 0)
  both_forward(pwm);
else if (pid_out > 0)
  both_reverse(pwm);
else
  both_motorOff();

old_error = error;
old_error_en = error_en;
```

```
      old_theta = theta;
    // Check if the characteristic was written
    if (customCharacteristic.written()) {
     // Get the length of the received data
      int length = customCharacteristic.valueLength();

      // Read the received data
      const unsigned char* receivedData = customCharacteristic.value();

      // Create a properly terminated string
      char receivedString[length + 1]; // +1 for null terminator
      memcpy(receivedString, receivedData, length);
      receivedString[length] = '\0'; // Null-terminate the string

      // Print the received data to the Serial Monitor
      Serial.print("Received data: ");
      Serial.println(receivedString);

      if (strcmp((const char*)receivedString, "r") == 0) {
        turn_L = 1.5;
        turn_R = 1;
        desired_vel = 0.8;
      }
      else if (strcmp((const char*)receivedString, "l") == 0) {
        turn_L = 1.5;
        turn_R = 1;
        desired_vel = 1;
      }
      else if (strcmp((const char*)receivedString, "f") == 0) {
        turn_L = 1;
        turn_R = 1;
        desired_vel = -4.5;
      }
      else if (strcmp((const char*)receivedString, "b") == 0) {
        turn_L = 1;
        turn_R = 1;
        desired_vel = 6;
      }
      else if (strcmp((const char*)receivedString, "s") == 0) {
        desired_vel = 0;
        turn_L = 1;
        turn_R = 1;
        //kp_en = 0;
        offset_angle = 1.25;
      }
      else if (strcmp((const char*)receivedString, "p") == 0) {
```

```
        desired_vel +=1;
      }
      else if (strcmp((const char*)receivedString, "n") == 0) {
        desired_vel -=1;
      }
      else if (strcmp((const char*)receivedString, "kp") == 0) {
        kp_en += 0.1;
      }
      else if (strcmp((const char*)receivedString, "kn") == 0) {
        kp_en -= 0.1;
      }
      else if (strcmp((const char*)receivedString, "op") == 0) {
        offset_angle +=0.1;
      }
      else if (strcmp((const char*)receivedString, "on") == 0) {
        offset_angle -=0.1;
      }

      // Optionally, respond by updating the characteristic's value
      //customCharacteristic.writeValue("Data received");
      Serial.println(desired_angle);
    }
   }
  }
    digitalWrite(LED_BUILTIN, LOW); // Turn off LED when disconnected
    Serial.println("Disconnected from central.");
  }
}
```

Forward_Backward_Turn_Ramp.ino

```
#include <AS5600.h>
#include <ArduinoBLE.h>
#include <Wire.h>
#include "TCA9548A.h"
#include <math.h>
#include <Arduino_BMI270_BMM150.h>
#include <string.h>

#define BUFFER_SIZE 20

TCA9548A I2CMux;
AS5600 encoderLeft;
AS5600 encoderRight;

// Define a custom BLE service and characteristic
BLEService customService("00000000-5EC4-4083-81CD-A10B8D5CF6ED");
```

```
BLECharacteristic customCharacteristic(
    "00000001-5EC4-4083-81CD-A10B8D5CF6ED", BLERead | BLEWrite | BLENotify,
BUFFER_SIZE, false);



// Define motor pins
#define BIN1 D3
#define BIN2 D5
#define AIN2 D6
#define AIN1 D9
#define A 1
#define B 2

float vel, angV0,angV1;
// Constants and state variables
float k = 0.9;
float kp = 7;
float ki = 100;
float kd = 0.35;
float kp_en = 0;
float ki_en = kp_en/200.0;

float offset_angle = 0.3; // balance setpoint
float desired_angle = 0; // balance setpoint
float desired_vel = 0;
float turn_L = 1;
float turn_R = 1;
float old_theta = 0;
float gyro_theta = 0;
float old_error = 0;
float old_error_en = 0;
float i_theta = 0;
float i_theta_en = 0;

int task = 0;
String input;
unsigned long start_time = 0;

// Motor control functions
void motorOff(int num){
  if(num == A){
    analogWrite(AIN1,255);
    analogWrite(AIN2,255);
  } else if(num == B){
    analogWrite(BIN1,255);
```

```
    analogWrite(BIN2,255);
  }
}

void forward(int num, int pwm){
  if(num == A){
    analogWrite(AIN1,255);
    if(turn_L > 0)
      analogWrite(AIN2,constrain(int((255 - pwm)*turn_L), 0, 255));
    else if(turn_L == 1)
      analogWrite(AIN2,constrain(255 - pwm, 0, 255));
    else
      analogWrite(AIN2,constrain(int((255 - pwm)/(-turn_L)), 0, 255));
  }
  else if(num == B){
    analogWrite(BIN1,255);
    analogWrite(BIN2,constrain(int((255-pwm)*turn_R), 0, 255));
  }
}

void reverse(int num, int pwm){
  if(num == A){
    if(turn_L < 0)
      analogWrite(AIN1,constrain(int((255 - pwm)*(-turn_L)), 0, 255));
    else if(turn_L == 1)
      analogWrite(AIN1,constrain(255 - pwm, 0, 255));
    else
      analogWrite(AIN1,constrain(int((255 - pwm)/(turn_L)), 0, 255));
    analogWrite(AIN2,255);
  } else if(num == B){
    analogWrite(BIN1,constrain(int((255-pwm)*turn_R), 0, 255));
    analogWrite(BIN2,255);
  }
}

void both_motorOff(){
  motorOff(A);
  motorOff(B);
}

void both_forward(int pwm){
  forward(A, pwm);
  forward(B, int(pwm*0.98));
}

void both_reverse(int pwm){
```

```
  reverse(A, pwm);
  reverse(B, int(pwm*0.98));
}

void setup() {
  //Serial.begin(9600);
  //Serial.setTimeout(10);
  //while (!Serial);

  if (!IMU.begin()) {
    Serial.println("Failed to initialize IMU!");
    while (1);
  }
  Wire.begin();
  I2CMux.begin(Wire);
  I2CMux.closeAll();

  I2CMux.openChannel(0);
  delay(10);

  encoderLeft.begin(4);  //  set direction pin.
  encoderLeft.setDirection(AS5600_CLOCK_WISE);  //  default, just be explicit.
  encoderRight.begin(4);  //  set direction pin.
  encoderRight.setDirection(AS5600_CLOCK_WISE);  //  default, just be explicit.

  int b = encoderLeft.isConnected();
  Serial.print("Connect: ");
  Serial.println(b);

  int c = encoderRight.isConnected();
  Serial.print("Connect: ");
  Serial.println(c);

  pinMode(BIN1, OUTPUT);
  pinMode(BIN2, OUTPUT);
  pinMode(AIN1, OUTPUT);
  pinMode(AIN2, OUTPUT);

    // Initialize the built-in LED to indicate connection status
  pinMode(LED_BUILTIN, OUTPUT);

  if (!BLE.begin()) {
    Serial.println("Starting BLE failed!");
    while (1);
  }
```

```
  // Set the device name and local name
  BLE.setLocalName("Lance");
  BLE.setDeviceName("Lance");

  // Add the characteristic to the service
  customService.addCharacteristic(customCharacteristic);

  // Add the service
  BLE.addService(customService);

  // Set an initial value for the characteristic
  customCharacteristic.writeValue("Waiting for data");

  // Start advertising the service
  BLE.advertise();

  Serial.println("BluetoothÂ® device active, waiting for connections...");
  Serial.println("Setup Finished");
  start_time = micros();
}

void keyboard_test (void) {
    if (Serial.available() > 0) {
        input = Serial.readString();
        input.trim();
        input.toLowerCase();
    }

    if (input == "kp") {
        task = 1;
    }
    else if (input == "ki") {
        task = 2;
    }
    else if (input == "kd") {
        task = 3;
    }
    else if (input == "reset") {
        i_theta = 0;
    }
    else if (input == "kpen") {
      task = 4;
    }
     else if (input == "angle") {
      task = 5;
    }
```

```
    switch (task) {
        case 1:
            if(input == "0" || input.toFloat() > 0) kp = input.toFloat();
        break;
        case 2:
            if(input == "0" || input.toFloat() > 0) ki = input.toFloat();
        break;
        case 3:
            if(input == "0" || input.toFloat() > 0) kd = input.toFloat();
        break;
        case 4:
            if(input == "0" || input.toFloat() > 0) kp_en = input.toFloat();
        break;
        case 5:
            if(input == "0" || input.toFloat() > 0) offset_angle = input.toFloat();
        break;
    }
}

void loop() {
  float acc_x, acc_y, acc_z;
  float gyro_x, gyro_y, gyro_z;
  float acc_theta, theta, vel;
  float error, error_en, d_theta, pid_out, pid_out_imu, pid_out_en;
  int pwm;
  //Bluetooth setup
  // Wait for a BLE central to connect
  BLEDevice central = BLE.central();

  if (central) {
    //Serial.print("Connected to central: ");
    Serial.println(central.address());
    digitalWrite(LED_BUILTIN, HIGH); // Turn on LED to indicate connection

    // Keep running while connected
    while (central.connected()) {
      keyboard_test();
      //Serial.println("Connected from central.");
      //Finish bluetooth setup
        if (IMU.accelerationAvailable() && IMU.gyroscopeAvailable()) {
          // Read accelerometer
          IMU.readAcceleration(acc_x, acc_y, acc_z);
          acc_theta = atan2(acc_y, acc_z) * 180.0 / M_PI;
```

```
// Read gyroscope
IMU.readGyroscope(gyro_x, gyro_y, gyro_z);
gyro_x = -gyro_x;

// Calculate delta time
unsigned long now = micros();
float dt = (now - start_time) / 1000000.0;
start_time = now;

// Integrate gyro
gyro_theta += gyro_x * dt;

// Complementary filter
theta = (1 - k) * acc_theta + k * (old_theta + gyro_x * dt);

//Angular speed
angV0 =encoderLeft.getAngularSpeed(AS5600_MODE_RPM);
Serial.print("left encoder speed: ");
Serial.println(angV0);
I2CMux.closeChannel(0);
I2CMux.openChannel(1);
angV1 = encoderRight.getAngularSpeed(AS5600_MODE_RPM);
Serial.print("right encoder speed: ");
Serial.println(angV1);
I2CMux.closeChannel(1);
I2CMux.openChannel(0);
vel =(angV0 - angV1)/2;//in angle/sec
Serial.print("angular speed: ");
Serial.println(vel);
vel = vel * PI / 180 * 4;// in cm/sec
Serial.print("velocity: ");
Serial.println(vel);

//displacement into a capping(?) constant
error_en = desired_vel - vel;
Serial.print("velocity desired: ");
Serial.println(desired_vel);

Serial.print("velocity error: ");
Serial.println(error_en);
i_theta_en += ki_en *(error_en + old_error_en) / 2.0 * dt;
Serial.print("kp_en: ");
Serial.println(kp_en);
Serial.print("i theta_en: ");
Serial.println(i_theta_en);
pid_out_en = (kp_en * error_en) + constrain(i_theta_en,-10,10);
```

```
      Serial.print("output desired angle:  ");
      Serial.println(pid_out_en);
      desired_angle = constrain(pid_out_en,-15,15);

      // PID calculations
      error = (offset_angle + desired_angle) - theta;
      d_theta = (old_theta - theta) / dt;
      i_theta += ki *(error + old_error) / 2.0 * dt;
      i_theta = constrain(i_theta,-50,50);
      pid_out = (kp * error) + (i_theta) + (kd * d_theta);
      if(pid_out < -255 or pid_out > 255) {
        pwm = 255;
      }
      else pwm = int(abs(pid_out));

      //Serial.print("offset_angle: "); Serial.print(offset_angle);
      //Serial.print("\t");
      //Serial.print("Kp: "); Serial.print(kp);
      //Serial.print("\t");
      //Serial.print("Ki: "); Serial.print(ki);
      //Serial.print("\t");
      //Serial.print("Kd: "); Serial.print(kd);
      //Serial.println("\t");
      //Serial.print(" i_theta: "); Serial.print(i_theta);
      //Serial.print("\t");
      //Serial.print(" kd*d_theta: "); Serial.println(kd*d_theta);

      // Apply motor control
      if (pid_out < 0)
        both_forward(pwm);
      else if (pid_out > 0)
        both_reverse(pwm);
      else
        both_motorOff();

      old_error = error;
      old_error_en = error_en;
      old_theta = theta;
    // Check if the characteristic was written
    if (customCharacteristic.written()) {
     // Get the length of the received data
      int length = customCharacteristic.valueLength();

      // Read the received data
      const unsigned char* receivedData = customCharacteristic.value();
```

```
// Create a properly terminated string
char receivedString[length + 1]; // +1 for null terminator
memcpy(receivedString, receivedData, length);
receivedString[length] = '\0'; // Null-terminate the string

// Print the received data to the Serial Monitor
Serial.print("Received data: ");
Serial.println(receivedString);

if (strcmp((const char*)receivedString, "l") == 0) {
  turn_L = 1.1;
  turn_R = 1;
  kp_en = 0.1;
}
else if (strcmp((const char*)receivedString, "r") == 0) {
  turn_L = -1.15;
  turn_R = 1;
  kp_en = 0.1;
}
else if (strcmp((const char*)receivedString, "f") == 0) {
  turn_L = 1;
  turn_R = 1;
  desired_vel = 5;
  kp_en = -0.1;
}
else if (strcmp((const char*)receivedString, "b") == 0) {
  turn_L = 1;
  turn_R = 1;
  desired_vel = -5;
  kp_en = -0.1;
}
else if (strcmp((const char*)receivedString, "s") == 0) {
  desired_vel = 0;
  turn_L = 1;
  turn_R = 1;
  kp_en = 0;
  desired_angle = 0.3;
}
else if (strcmp((const char*)receivedString, "p") == 0) {
  desired_vel +=0.1;
}
else if (strcmp((const char*)receivedString, "n") == 0) {
  desired_vel -=0.1;
}
else if (strcmp((const char*)receivedString, "kp") == 0) {
  kp_en += 0.1;
```

```
      }
      else if (strcmp((const char*)receivedString, "kn") == 0) {
        kp_en -= 0.1;
      }
      else if (strcmp((const char*)receivedString, "op") == 0) {
        offset_angle +=0.1;
      }
      else if (strcmp((const char*)receivedString, "on") == 0) {
        offset_angle -=0.1;
      }
      else if (strcmp((const char*)receivedString, "ufp") == 0) {
        i_theta = 0;
        i_theta_en = 0;
      }
      else if (strcmp((const char*)receivedString, "dfn") == 0) {
        offset_angle -=2;
      }
      else if (strcmp((const char*)receivedString, "dfp") == 0) {
        offset_angle +=2;
      }

      // Optionally, respond by updating the characteristic's value
      //customCharacteristic.writeValue("Data received");
      Serial.println(desired_angle);
    }
   }
  }
    digitalWrite(LED_BUILTIN, LOW); // Turn off LED when disconnected
    Serial.println("Disconnected from central.");
  }
}
```

bluetooth_app.dart

```
import 'dart:convert';
import 'dart:async';
import 'package:flutter/material.dart';
import 'package:flutter_reactive_ble/flutter_reactive_ble.dart';

// define UUIDs as constants - these should match the Arduino code
const String serviceUUID = "00000000-5EC4-4083-81CD-A10B8D5CF6ED";
const String characteristicUUID = "00000001-5EC4-4083-81CD-A10B8D5CF6ED";

class MyHomePage extends StatefulWidget {
  const MyHomePage({super.key, required this.title});

  final String title;
```

```
  @override
  State<MyHomePage> createState() => _MyHomePageState();
}

class _MyHomePageState extends State<MyHomePage> {
  final _ble = FlutterReactiveBle();

  StreamSubscription<DiscoveredDevice>?
      _scanSub; // subscribe to bluetooth scanning stream
  StreamSubscription<ConnectionStateUpdate>?
      _connectSub; // subscribe to bluetooth connection stream
  StreamSubscription<List<int>>? _notifySub;

  List<DiscoveredDevice> _devices = [];
  String? _selectedDeviceId; // will hold the device ID selected for connection
  String?
      _selectedDeviceName; // will hold the device name selected for connection
  var _stateMessage = 'Scanning...'; // displays app status
  QualifiedCharacteristic? _writeCharacteristic;

  bool _isConnected = false; // flag to indicate connection

  // on initialization scan for devices
  Timer? _scanTimer;

  @override
  void initState() {
    super.initState();
    _scanSub = _ble.scanForDevices(withServices: []).listen(_onScanUpdate);
    _scanTimer = Timer.periodic(Duration(seconds: 5), (timer) {
      _scanSub?.cancel();
      _scanSub = _ble.scanForDevices(withServices: []).listen(_onScanUpdate);
    });
  }

  // when terminating cancel all the subscriptions
  @override
  void dispose() {
    _notifySub?.cancel();
    _connectSub?.cancel();
    _scanSub?.cancel();
    super.dispose();
  }

  // update devices that found with "BLE" in their name
```

```
  void _onScanUpdate(DiscoveredDevice d) {
    if (d.name.contains("Lance") &&
        !_devices.any((device) => device.id == d.id)) {
      setState(() {
        _devices.add(d);
      });
    }
  }

  // Connect to the devices that was selected by user
  void _connectToDevice() {
    if (_selectedDeviceId != null) {
      setState(() {
        _stateMessage = 'Connecting to $_selectedDeviceName...';
      });

      _connectSub = _ble.connectToDevice(id: _selectedDeviceId!).listen(
        (update) {
          if (update.connectionState == DeviceConnectionState.connected) {
            setState(() {
              _stateMessage = 'Connected to $_selectedDeviceName!';
              _isConnected = true;
            });
            _onConnected(_selectedDeviceId!);
          }
        },
        onError: (error) {
          setState(() {
            _stateMessage = 'Connection error: $error';
          });
        },
      );
    }
  }

  // Handle disconnection
  void _disconnectFromDevice() {
    try {
      if (_notifySub != null) {
        _notifySub?.cancel();
        _notifySub = null;
      }

      if (_connectSub != null) {
        _connectSub?.cancel();
        _connectSub = null;
```

```
    }

    setState(() {
      _isConnected = false;
      _stateMessage = 'Disconnected from $_selectedDeviceName.';
      _writeCharacteristic = null;
    });
  } catch (e) {
    setState(() {
      _stateMessage = 'Error during disconnection: $e';
    });
  }
}

void _onConnected(String deviceId) {
  final characteristic = QualifiedCharacteristic(
    deviceId: deviceId,
    serviceId: Uuid.parse(serviceUUID), // Use the constant here
    characteristicId: Uuid.parse(characteristicUUID), // Use the constant here
  );

  _writeCharacteristic = characteristic;

  _notifySub = _ble.subscribeToCharacteristic(characteristic).listen((bytes) {
    setState(() {
      _stateMessage = 'Data received: ${Utf8Decoder().convert(bytes)}';
    });
  });
}

Future<void> _sendCommand(String command) async {
  if (_writeCharacteristic != null) {
    try {
      await _ble.writeCharacteristicWithResponse(
        _writeCharacteristic!,
        value: utf8.encode(command),
      );
      setState(() {
        _stateMessage = "Command '$command' sent!";
      });
    } catch (e) {
      setState(() {
        _stateMessage = "Error sending command: $e";
      });
    }
  }
```

```
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
        title: Text(widget.title),
      ),
      body: Column(
        children: [
          Container(
            padding: const EdgeInsets.all(16.0),
            color: Colors.grey[300],
            width: double.infinity,
            child: Text(
              _stateMessage,
              style: Theme.of(context).textTheme.titleMedium,
              textAlign: TextAlign.center,
            ),
          ),
          if (_devices.isNotEmpty)
            Padding(
              padding: const EdgeInsets.all(16.0),
              child: DropdownButton<String>(
                isExpanded: true,
                hint: const Text("Select a BLE Device"),
                value: _selectedDeviceId,
                items: _devices.map((device) {
                  return DropdownMenuItem(
                    value: device.id,
                    child: Text(device.name),
                  );
                }).toList(),
                onChanged: (value) {
                  setState(() {
                    _selectedDeviceId = value;
                    _selectedDeviceName = _devices
                        .firstWhere((device) => device.id == value)
                        .name;
                  });
                },
              ),
            ),
          if (!_isConnected)
            ElevatedButton(
```

```
            onPressed: _selectedDeviceId != null ? _connectToDevice : null,
            child: const Text('Connect'),
          ),
        if (_isConnected)
          ElevatedButton(
            onPressed: _disconnectFromDevice,
            child: const Text('Disconnect'),
          ),
        // ***************** command buttons *****************
        Expanded(
          child: Column(
            mainAxisAlignment: MainAxisAlignment.center,
            children: [
              // Joystick Buttons
              Row(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  ElevatedButton(
                    onPressed:
                        _isConnected ? () => _sendCommand('f') : null,
                    child: const Icon(Icons.arrow_upward),
                  ),
                ],
              ),
              const SizedBox(height: 10),
              Row(
                mainAxisAlignment: MainAxisAlignment.center,
                children: [
                  ElevatedButton(
                    onPressed:
                        _isConnected ? () => _sendCommand('l') : null,
                    child: const Icon(Icons.arrow_back),
                  ),
                  const SizedBox(width: 10),
                  ElevatedButton(
                    onPressed:
                        _isConnected ? () => _sendCommand('b') : null,
                    child: const Icon(Icons.arrow_downward),
                  ),
                  const SizedBox(width: 10),
                  ElevatedButton(
                    onPressed:
                        _isConnected ? () => _sendCommand('r') : null,
                    child: const Icon(Icons.arrow_forward),
                  ),
                ],
```

```
            ),
            const SizedBox(height: 20),

            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('s') : null,
                  child: const Text('STOP'),
                ),
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('p') : null,
                  child: const Text('v+1'),
                ),
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('n') : null,
                  child: const Text('v-1'),
                ),
                const SizedBox(width: 10),
              ],
            ),
            const SizedBox(height: 30),
            Row(
              mainAxisAlignment: MainAxisAlignment.center,
              children: [
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('kn') : null,
                  child: const Text('en-'),
                ),
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('kp') : null,
                  child: const Text('en+'),
                ),
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('on') : null,
                  child: const Text('angle-2'),
                ),
                const SizedBox(width: 10),
                ElevatedButton(
                  onPressed: _isConnected ? () => _sendCommand('op') : null,
                  child: const Text('angle+2'),
```

```
            ),
          ],
        ),
        const SizedBox(height: 40),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('ufp') : null,
              child: const Text('up+0.1'),
            ),
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('ufn') : null,
              child: const Text('up-'),
            ),
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('dfp') : null,
              child: const Text('dn+'),
            ),
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('dfn') : null,
              child: const Text('dn-'),
            ),
          ],
        ),
        const SizedBox(height: 50),
        Row(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('mrap') : null,
              child: const Text('min angle+0.5'),
            ),
            const SizedBox(width: 10),
            ElevatedButton(
              onPressed: _isConnected ? () => _sendCommand('mran') : null,
              child: const Text('min angle-0.5'),
            ),
          ],
        ),
      ],
```

```
            ),
          ),
          // ***************** end of command buttons ****************
        ],
      ),
    );
  }
}
```